Advanced Object Oriented Programming

Xingjie ZHANG

아주대학교
AJOU UNIVERSITY

아주대학교
AJOU UNIVERSITY

Inheritance and polymorphism

Instructor: Xingjie ZHANG

Email : xzhang@ajou.ac.kr

Wechat : runningSnaile

CONTENTS

01 Inheritance

02 Inheritance and constructors

03 Overriding

04 Unified Modeling Language

05 Polymorphism

아주대학교 AJOU UNIVERSITY

01

Inheritance

Inheritance

- A syntax for reusing (extending) object-level code.
- Can be explained using an **is-a** relationship.
- Parent/child classes, base(基类)/derived(派生类) classes.
- Java does not allow multiple inheritance.
- In Java, all classes are derived from **Object**.

```
class child extends parent
```

Inheritance

- When creating a derived class, the base class is also created (the constructor is called).
- Derived classes can access base class members.
- Base class methods can be overridden in derived classes.
- Abstract data type concepts can be applied.
- When inheritance relationships are applied, the code flow tends to become two-dimensional and hidden.

Inheritance

- _inheritSample

```
class MyTest {
    private int parentData = 10;
    public int getData() {
        return parentData;
    }
    public void printData() {
        System.out.println(parentData);
    }
}
```

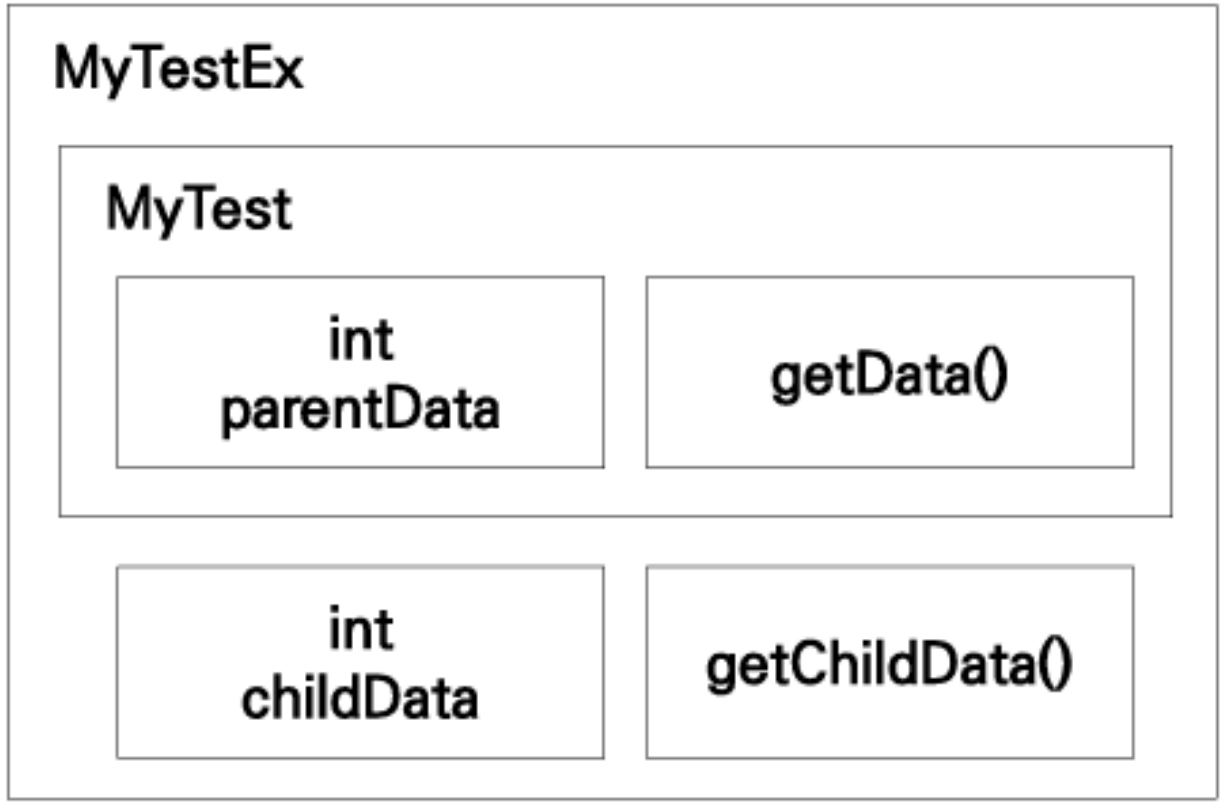
Inheritance

```
class MyTestEx extends MyTest {
    private int childData = 20;
    public int getChildData() {
        return childData;
    }
}

public class Main {
    public static void main(String[] args) {
        MyTestEx t = new MyTestEx();
        System.out.println(t.getData());
        System.out.println(t.getChildData());
    }
}
```

```
10
20
```

Inheritance

- Instance memory structure in inheritance relationship



02

Inheritance and constructors

Inheritance and constructors

- **Derived class constructors are called first but executed last.**
- Similar to recursive calls, constructor functions are stacked upward.
- **Defining base class fields in derived classes is often inappropriate.**

Inheritance and constructors

- Accessing base classes from derived classes

```
class MyTest {
    private int parentData = 10;
    public int getData() {
        return parentData;
    }
    ...
}

class MyTestEx extends MyTest {
    ...
    public int getParentData() {
        return parentData;
    }
}
```

```
java: parentData has private access in MyTest
```

Inheritance and constructors

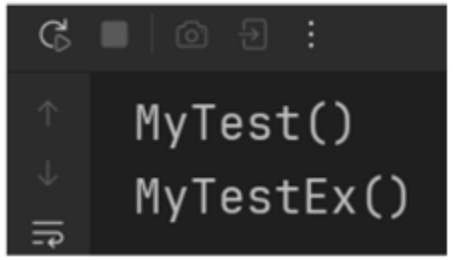
- Accessing base classes from derived classes

```
class MyTest {
    protected int parentData = 10;
    public int getData() {
        return parentData;
    }
    public void printData() {
        System.out.println(parentData);
    }
}
```

Inheritance and constructors

- _constructorCallStack

```
class MyTest {
    private int parentData = 10;
    MyTest() {
        System.out.println("MyTest()");
    }
    ...
}

class MyTestEx extends MyTest {
    private int childData = 20;
    MyTestEx() {
        System.out.println("MyTestEx()");
 ...
}
```



Inheritance and constructors

- When the parent class and derived class are defined
  - The "present" definition of the parent class determines that the derived class will be defined "in the future.“
- Problems due to lack of understanding
  - The authors of the parent class and derived class may be different.
- **Write the current class code with future derived classes in mind.**

Inheritance and constructors

- Defining base class fields in derived classes is often inappropriate.

```
class Test {
    protected byte[] string;
    Test() {
        string = new byte[]{'h', 'e', 'l', 'l', 'o'};
    }
    void setString(String param) {
        string = param.getBytes().clone();
    }
    String getString() {
        String str = new String(string);
        return str;
    }
}
```

Inheritance and constructors

- Defining base class fields in derived classes is often inappropriate.

```
class TestEx extends Test {
    TestEx() {
        string = null;
    }
}

public class Main {
    public static void main(String[] args) {
        Test t = new TestEx();
        System.out.println(t.getString());
    }
}
```

Inheritance and constructors

- Selecting a Constructor from a Parent Class
  - Forcing a derived class constructor to call a desired constructor from among multiple constructors defined in the parent class.
  - **super()**

Inheritance and constructors

- _superSample

```
class Test {
    public Test () {
        System.out.println("Test()");
    }
    public Test(int param) {
        System.out.println("Test(int)");
    }
}
class TestEx extends Test {
    public TestEx() {
        System.out.println("TestEx()");
    }
    public TestEx(int param) {
        System.out.println("TestEx(int)");
    }
}
```

```
Test()
TestEx(int)
```

Inheritance and constructors

- Change the call to a different constructor!

```
class TestEx extends Test {
    public TestEx() {
        System.out.println("TestEx()");
    }
    public TestEx(int param) {
        super(param);
        System.out.println("TestEx(int)");
    }
}
```

```
Test(int)
TestEx(int)
```

03

Overriding

Overriding

- Base class methods can be overridden in derived classes.
  - **Note:** This differs from multiple definitions.
  - **@Override** annotation
- Overriding a base class method can be seen as replacing an existing method or adding code.
- When overriding a method, the actual instance type takes precedence.
- The final declaration prevents a method from being overridden in a derived class.
  - **public final void getData()**

Overriding

- _overrideSample

```
class Test {
    protected int data;
    public void setData(int param) {
        System.out.println("Test.setData()");
        this.data = param;
    }
}
class TestEx extends Test {
    public void setData(int param) {
        System.out.println("TestEx.setDataEx()");
        this.data = param;
    }
}
```

Overriding

```
public class Main {
    public static void main(String[] args) {
        Test t1 = new TestEx();
        TestEx t2 = new TestEx();
        t1.setData(5);
        t2.setData(5);
    }
}
```

Overriding

- Change the way you add code

```
class Test {
    protected int data;
    public void setData(int param) {
        System.out.println("Test.setData()");
        this.data = param;
    }
    public int getData() {
        return data;
    }
}
```

Overriding

- Change the way you add code

```
class TestEx extends Test {
    public void setData(int param) {
        System.out.println("TestEx.setDataEx()");
        if(param < 0)
            param = 0;
        super.setData(param);
    }
}

public class Main {
    public static void main(String[] args) {
        TestEx t = new TestEx();
        t.setData(-5);
        System.out.println(t.getData());
    }
}
```

```
TestEx.setDataEx()
Test.setData()
0
```

Called by Framework

- Methods redefined in derived classes can be called based on function call relationships established in the parent class.
- You can participate by extending the flow defined by the structure and adding your own code within it.
  - JSP, Pytorch, Tensorflow

Called by Framework

- _calledByFramework

```
class Test {
    protected int data;
    public void setData(int param) {
        System.out.println("Test.setData()");

        if(onSetData(param))
            this.data = param;
        else
            this.data = 0;
    }

    protected boolean onSetData(int param) {
        System.out.println("Test.onSetData()");
        return true;
    }
    ...
}
```

Called by Framework

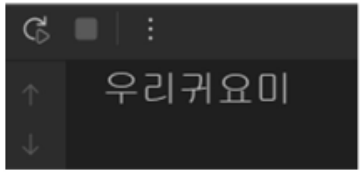
```
class TestEx extends Test {
    //Called by framework
    protected boolean onSetData(int param) {
        System.out.println("TestEx.onSetData()");
        if(param > 100 || param < 0)
            return false;

        return true;
    }
}

public class Main {
    public static void main(String[] args) {
        TestEx t = new TestEx();
        t.setData(-5);
        System.out.println(t.getData());
    }
}
```

```
Test.setData()
TestEx.onSetData()
0
```

Called by Framework

- Practices
  - 在MyString()类里定义onSetString()函数，并在setString()函数中调用
  - 定义派生类MyStringEx,重写onSetString函数，防止特定字符串无法赋值进来

```
public class Main {
    public static void main(String[] args) {
        MyStringEx str = new MyStringEx();
        str.setString("멍멍이아들");
        System.out.println(str.getString());
    }
}
```



04

Unified Modeling Language

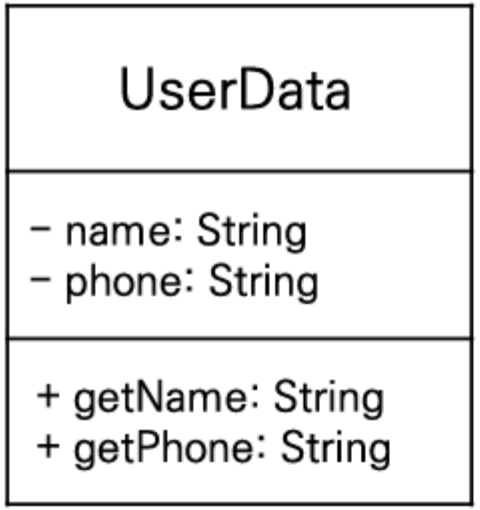
Relationships between objects

- A class can have other classes as fields
  - **Composition, Aggregation**
- Inheritance relationship
  - Dependencies arise from the very establishment of the class.
- Limited dependencies on specific functions.
- Class design defines the object and relationships that achieve a specific goal.
  - Typically described in **UML**

Unified Modeling Language (UML)

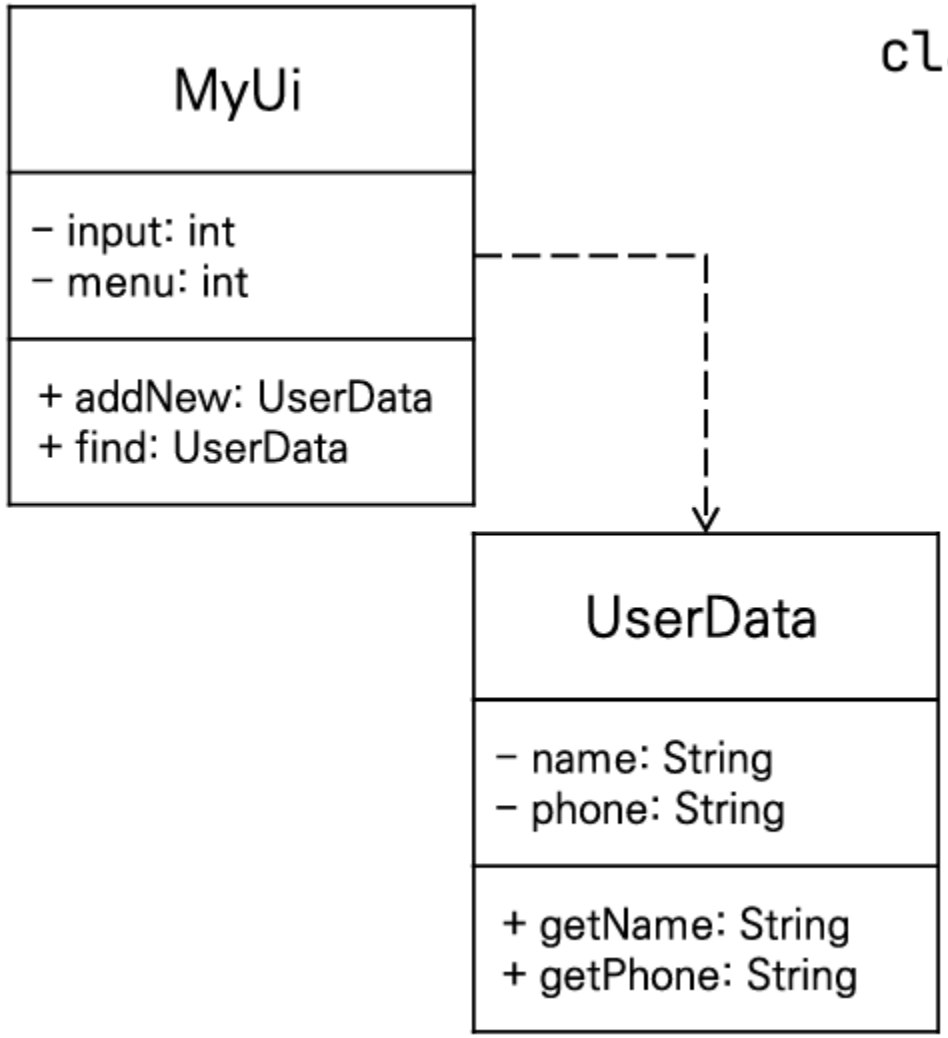
- A methodology for modeling classes and associations using standard modeling languages
  - **Class diagram**
  - **Sequence diagram**

Class diagram



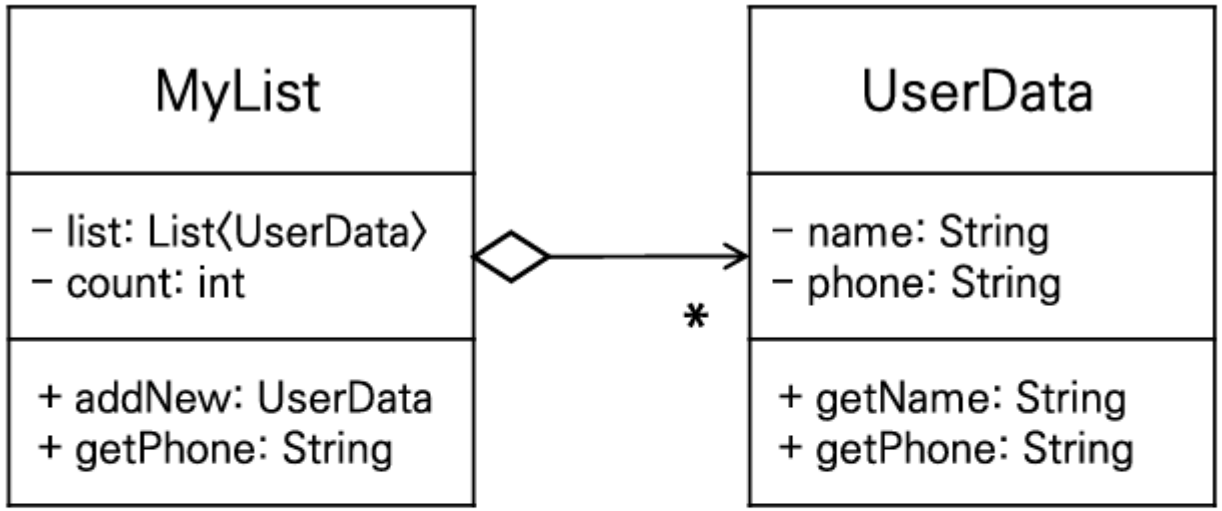

```
class UserData {
    private String name;
    private String phone;

    public String getName() {
        return name;
    }

    public String getPhone() {
        return phone;
    }
}
```

Relationships between Class



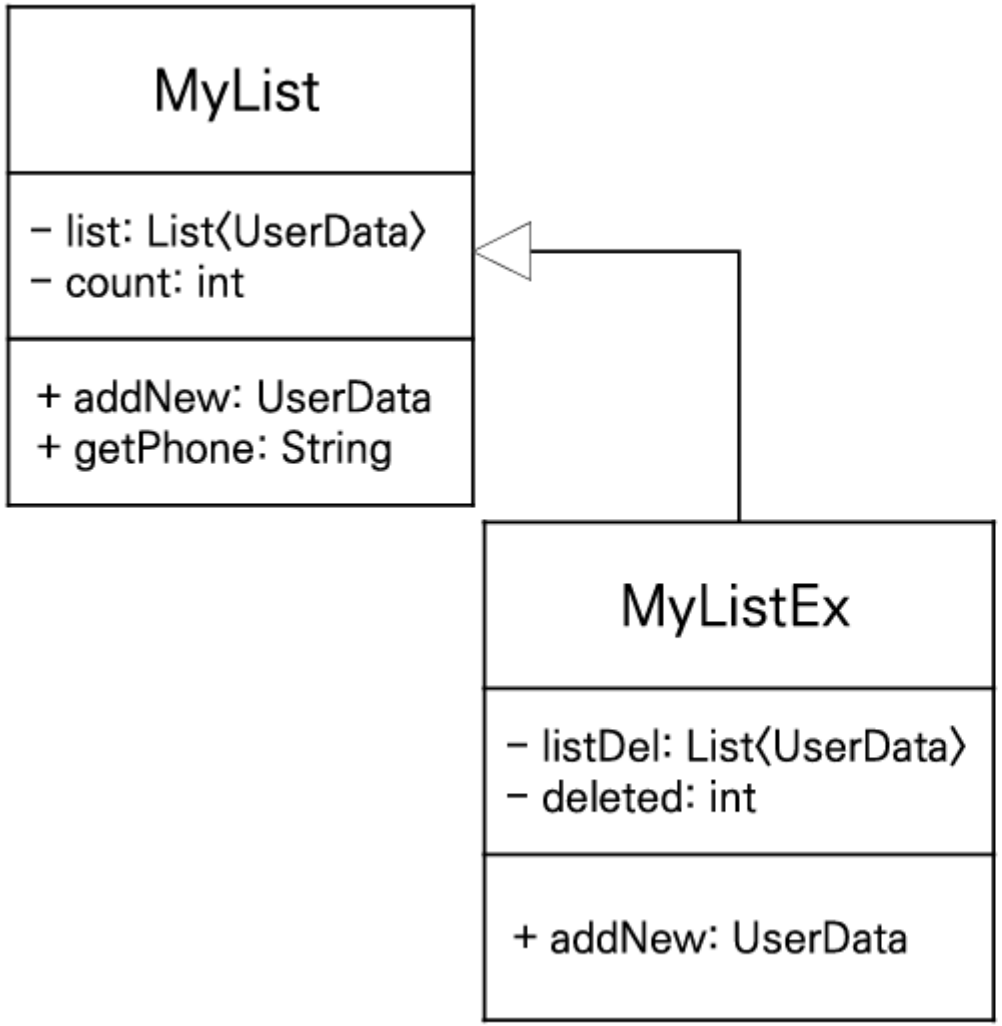

```
class MyUi {
    private int input;
    private int menu;

    public UserData addNew() {
        return new UserData();
    }

    public UserData find(UserData user)
    {
        return new UserData();
    }
}
```

Relationships between Class



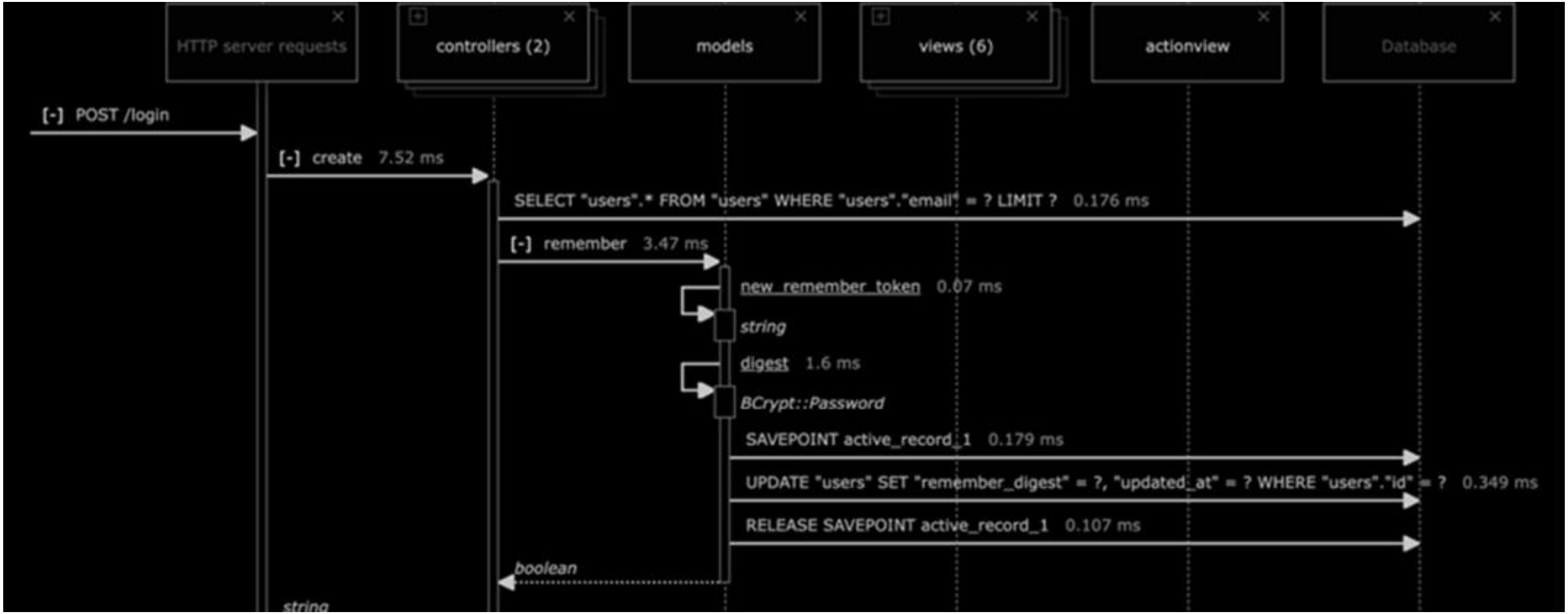

```
class MyList {
    private List<UserData> list;
}
```

Relationships between Class




```
class MyList {
    private List<UserData> list;

    UserData addNew() {
        return new UserData();
    }
}

class MyListEx extends MyList {
    private List<UserData> listDel;
    private int deleted;

    @Override
    UserData addNew() {
        return new UserData();
    }
}
```

Sequence diagram



05

Polymorphism

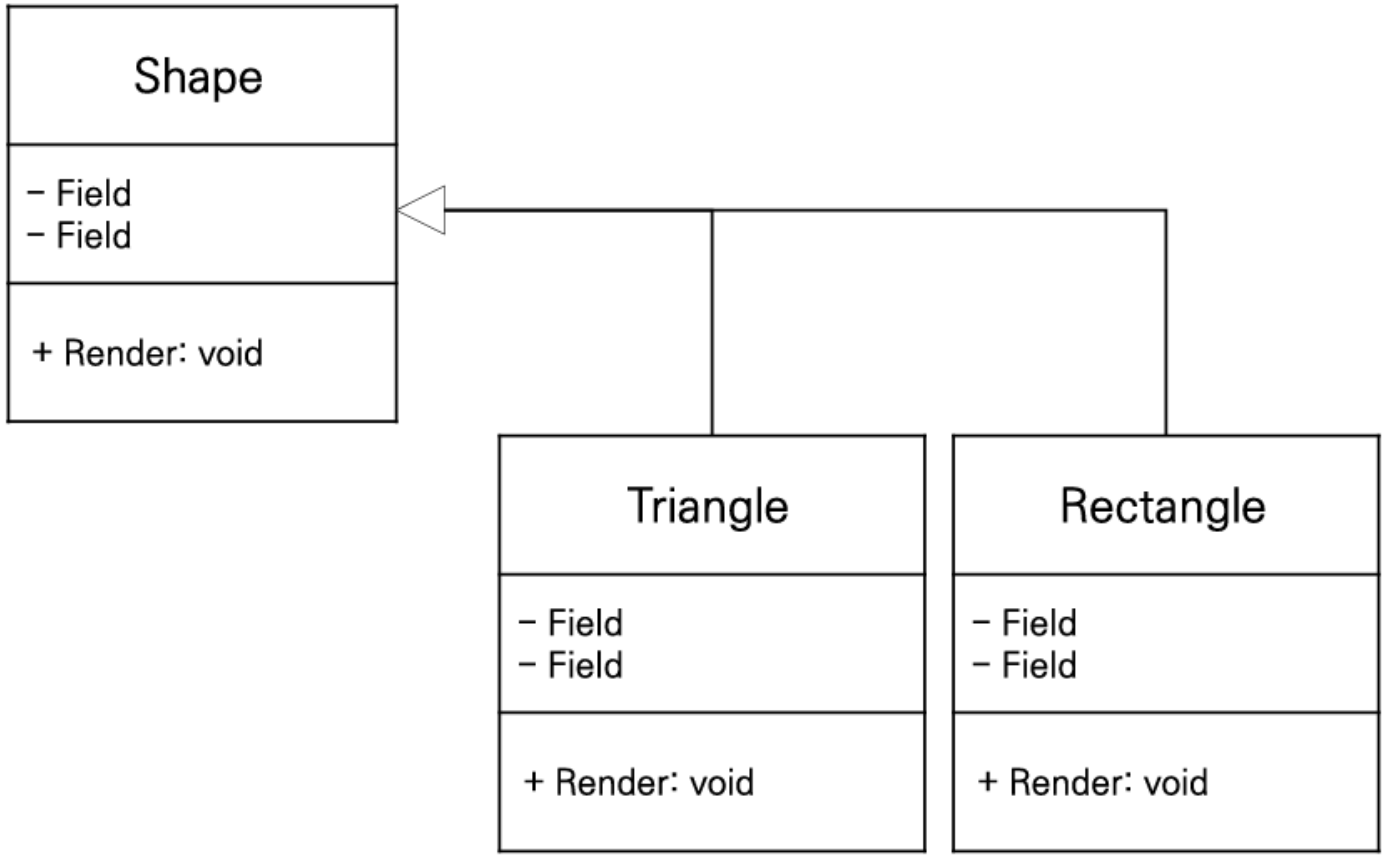
Polymorphism

- The dictionary definition is: "The phenomenon of organisms of the same species exhibiting diverse forms or characteristics."
- A key characteristic of object-oriented programming:
  - **Abstraction** (or generalization), concretization.
  - Structures can be simplified at a high level of abstraction
  - **The principles by which frameworks can be implemented**

- **Upcasting**: All derived types of a parent class can be cast to the parent class.

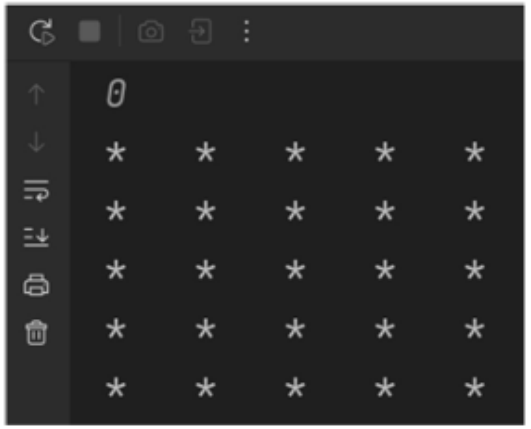
Polymorphism

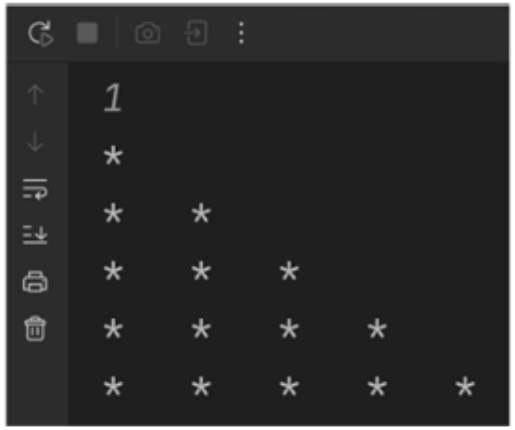
- Ex) Shape Class



Polymorphism

```
public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int input = s.nextInt();

        Shape shape;
        if(input == 0)
            shape = new Rectangle();
        else
            shape = new Triangle();

        shape.render();
    }
}
```





Polymorphism

- **Downcasting**: Casting the target instance pointed to by a parent class type reference to a specific derived type.
- Be careful about inverting dependencies.

- **instanceof operator**:

```
boolean result = Obj instanceof instance;
```

Polymorphism

- _downcastSample

```
Scanner s = new Scanner(System.in);
int input = s.nextInt();

Shape shape;
if(input == 0)
    shape = new Rectangle();
else
    shape = new Triangle();

shape.render();
if(shape instanceof Rectangle)
    System.out.println("Rectangle clss object");
else
    System.out.println("Triangle clss object");
```

Abstract class

- Instances cannot be created using the new operation.
  - C++ pure virtual class concept.
- Classes that exist solely for derived classes.
- Exists from a design perspective.
- It's common to have abstract methods.
  - **abstract void getA();**
  - Must be overridden in derived types.

```
public abstract class class-name
```

Abstract class

- _abstractSample

```
abstract class Shape {
    abstract public void render();
}

class Triangle extends Shape {
    @Override
    public void render() {
        for(int i = 0; i < 5; ++i) {
            for(int j = 0; j < i + 1;  ++j)
                System.out.print("*\t");
            System.out.print('\n');
        }
    }
}
```

Interface

- It can be understood as an abstract class without fields.
  - **It can have constant fields.**
- For interface derivation, use "**implements**" instead of "**extends**."
- Interfaces are pure declarations that describe method prototypes and do not have concrete definitions.
  - Definitions various symbolic constants.
  - Similar to header files in the C language.
  - Multiple inheritance is possible.

```
[public] interface iface-name
```

Interface

- _interfaceSample

```
interface Shape {
    public void render();
}

class Triangle implements Shape {
    @Override
    public void render() {
        for(int i = 0; i < 5; ++i) {
            for(int j = 0; j < i + 1;  ++j)
                System.out.print("*\t");
            System.out.print('\n');
        }
    }
}
```

Enumeration

- A syntax for defining symbolic constants provides type safety and allows for more readable code.
- A derived class of **java.lang.Enum**

Enumeration

- _enumSample

```
enum Week {
    MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY, SUNDAY
}

public class Main {
    public static void main(String[] args) {
        Week week = Week.SUNDAY;
        System.out.println(Week.valueOf("MONDAY").ordinal());
        System.out.println(week.name());
        System.out.println(week.ordinal());

        for(Week w : Week.values()) {
            System.out.println(w.name());
        }
    }
}
```

```
0
SUNDAY
6
MONDAY
TUESDAY
WEDNESDAY
THURSDAY
FRIDAY
SATURDAY
SUNDAY
```

Quiz

- 1. What is the main purpose of inheritance in object-oriented programming?
  A. To create new classes completely independently
  B. To increase code reusability and extensibility
  C. To hide complex relationships between classes
  D. To minimize memory usage

Quiz

- 2. When creating an instance of a child class in an inheritance relationship, what is the order of constructor execution?
  A. Child constructor → Parent constructor order for both call and execution.
  B. Parent constructor → Child constructor order for both call and execution.
  C. The child constructor calls the parent constructor, the parent constructor executes first, then the child constructor's code executes.
  D. Neither the call order nor the execution order is defined.

Quiz

- 3. Which access modifier should be used to allow a parent class member to be accessed by child classes but not by external classes?
  A. public
  B. private
  C. protected
  D. default (package-private)

Quiz

- 4. What does method overriding mean in object-oriented programming?
  A. Defining multiple methods with the same name
  B. Redefining a parent class method in a child class with the same name ✓
  C. Creating a method accessible only within the class
  D. Determining the method call at compile time

Quiz

- 5. When a parent class type reference variable points to a child class instance, which method is executed when calling an overridden method?
  A. The parent class method corresponding to the reference variable type
  B. The overridden method of the child class corresponding to the actual created object instance type
  C. The parent class method determined at compile time
  D. A runtime error occurs and the method does not execute

Quiz

- 6. What is the object-oriented characteristic where various child objects referenced by a parent type respond differently to the same method call?
  A. Inheritance
  B. Encapsulation
  C. Polymorphism
  D. Abstraction

Quiz

- 7. In Java, what type of casting cannot be guaranteed to always succeed when converting a parent class type to a child class type?
  A. Upcasting
  B. Downcasting
  C. Automatic type conversion
  D. Primitive type conversion

Quiz

- 8. What type of class cannot be instantiated directly and is mainly designed for child classes to complete its unfinished parts through inheritance?
  A. Regular class
  B. Final class
  C. Abstract class
  D. Static class

Quiz

- 9. In Java, what feature is not possible with classes but provides an effect similar to multiple inheritance by 'implementing' multiple parent type specifications?
  A. Abstract class
  B. Interface
  C. Enum
  D. Inner class

Quiz

- 10. What Java feature is primarily used to group related constant values under one name to improve code readability and stability?
  A. Array
  B. Collection
  C. Exception
  D. Enum

Q & A

THANK YOU